CODING · AULA 10 · 20 DE OUTUBRO DE 2026

A loja não esquece

Persistência e camadas. Os produtos passam a morar num banco de dados, e o código ganha uma organização que aguenta crescer.

Faculdade Senac Pernambuco · ADS 2026.2 · Prof. Sebastião Rogério

ONDE ESTAMOS NO SEMESTRE · 1 DE 3

O caminho até aqui

U1 · ✔ CONCLUÍDA

Fundamentos

11/08
Por que tantas linguagens

18/08
Paradigmas

25/08
Tipos e coleções

01/09
Funções, módulos e testes

U2 · ✔ CONCLUÍDA

Orientação a objetos

08/09
Classes

22/09
AV1 e encapsulamento

29/09
Herança, composição e polimorfismo

06/10
Contratos e a Loja de Roupas

U3 · ◀ ESTAMOS AQUI

Back-end e API

13/10
HTTP, REST e a primeira API

20/10
Hoje: persistência e camadas

U4

Front-end

27/10
HTML e CSS + ENTREGA 1

03/11
JavaScript e eventos

10/11
Integração com a API

U5

Entrega e avaliação

17/11
PROJETO FINAL

24/11
AV2

01/12
Framework (sem nota)

08/12
Encerramento

ONDE ESTAMOS NO SEMESTRE · 2 DE 3

A aplicação que estamos construindo, camada por camada

Telas no navegador	o que o cliente vê e clica · Aulas 11 a 13	falta
▲ fetch ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✔ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	◀ HOJE
▲ usa ▼		
Domínio: as classes	Produto, Carrinho, Promocao · Aulas 5 a 8	✔ pronto
▲ guarda ▼		
Banco de dados	os dados deixam de sumir · Aula 10	◀ HOJE

- Na Aula 9, a loja ganhou uma **porta**: a API.
- Hoje, os dados ganham uma **casa**: o banco.
- E entre a porta e a casa entram duas camadas que organizam o código: **serviço** e **repositório**.

ONDE ESTAMOS NO SEMESTRE · 3 DE 3

O que falta no semestre

DATA	AULA	TEMA	O QUE ACRESCENTA AO PROJETO
13/10	9	HTTP, REST e primeiros endpoints	a loja responde pela rede
20/10	**10**	**Persistência e camadas**	**hoje: os dados ficam guardados num banco**
27/10	11	HTML e CSS · **ENTREGA 1 DO PROJETO**	**a API da equipe, com dados guardados, testes e README**
03/11	12	JavaScript, DOM e eventos	as telas reagem ao usuário
10/11	13	Assincronismo e integração	as telas conversam com a API
17/11	14	**PROJETO FINAL**	apresentação com demonstração: 50% do segundo processo
24/11	15	**AV2** e devolutiva	prova: 50% do segundo processo
01/12 e 08/12	16 e 17	Framework e encerramento	conteúdo complementar, sem nota

OBJETIVOS DA AULA

Ao final, você consegue

1. Explicar por que dados em **memória** somem e o que é **persistência**.
2. Usar o **SQLite** pelo Python, com SQL parametrizado e **commit**.
3. Separar o código em **camadas**: rota, serviço, repositório e banco.
4. Mapear classes em tabelas com um **ORM** (SQLModel).
5. Transformar exceções do domínio em códigos **404, 409 e 422**.
6. Entregar um **CRUD** gravado no banco e testado com banco em memória.

O QUE VAMOS VER HOJE

Primeiro os conceitos, depois a prática

MUNDO 1

Por que persistir

teoria com exemplos

- A loja que esquece
- Banco relacional e SQLite
- SQL pelo Python

MUNDO 2

Camadas e ORM

teoria com exemplos

- Rota, serviço, repositório
- ORM e SQLModel
- Exceções e códigos HTTP

MUNDO 3

Prática na loja

vocês seguem a aula

- O repositório
- O serviço
- A API com CRUD

LABORATÓRIO

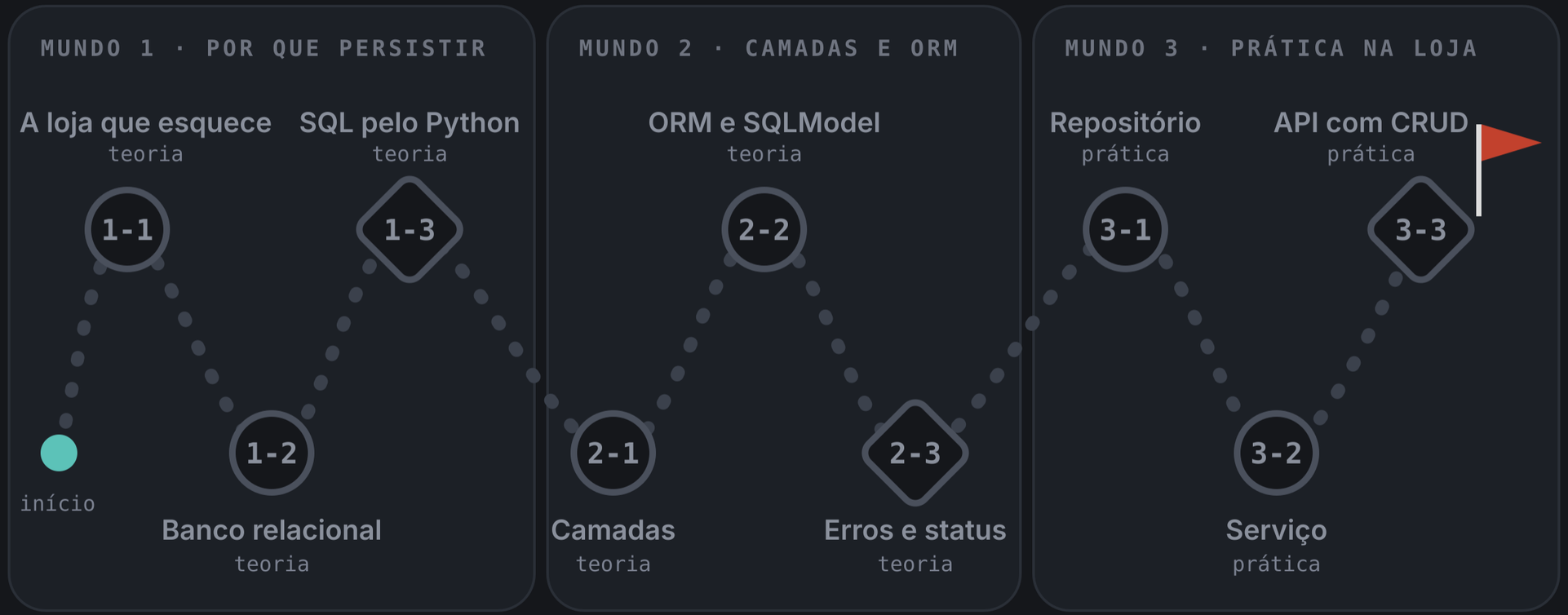
Projeto da equipe

em equipe

- CRUD de um recurso, gravado
- Checklist da Entrega 1

LEMBRETE

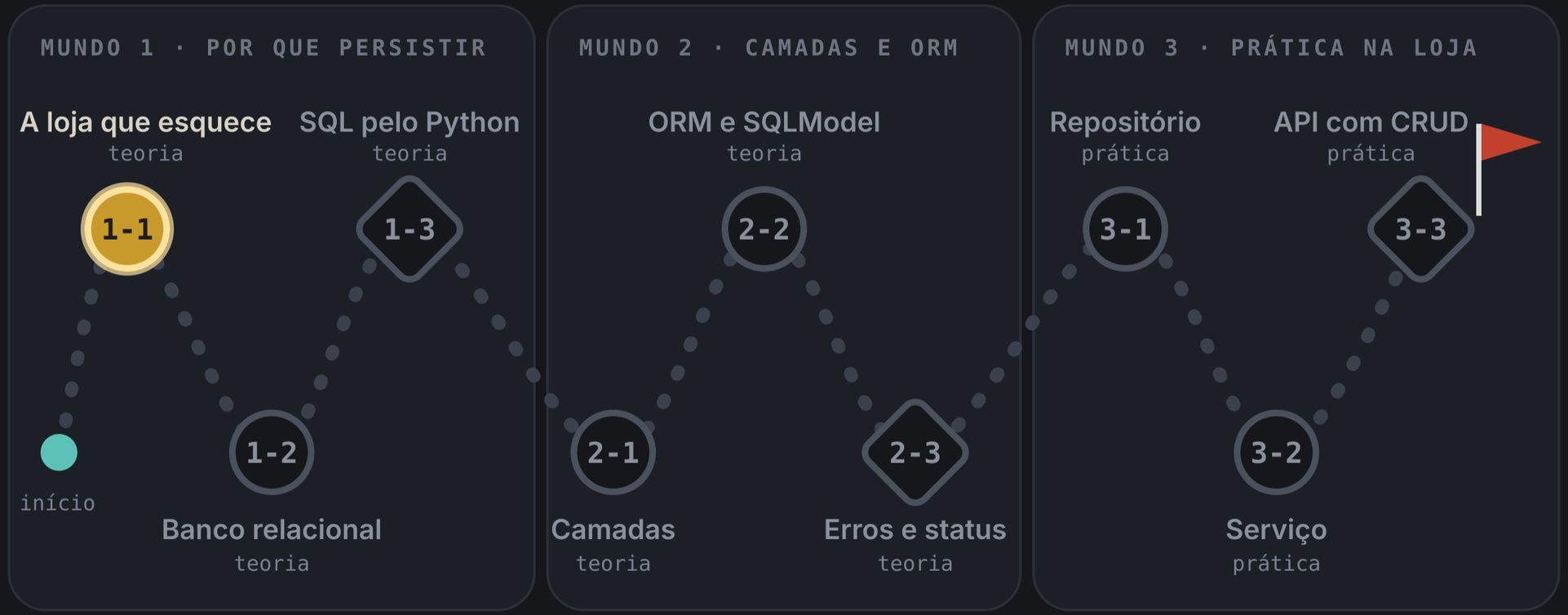
Entrega 1 do projeto na próxima terça, 27/10.



O MAPA DE HOJE

Nove fases, três mundos

Mundos 1 e 2: os conceitos, com exemplos. Mundo 3: a prática na Loja de Roupas, com checkpoints e o repositório gabarito.



FASE 1-1 · CONTEÚDO NOVO

A loja que esquece

O carrinho e os produtos da Aula 9 moram na memória do programa. Reiniciou, sumiu.

FASE 1-1 · PARA COMEÇAR

Na Aula 9, bastava salvar o api.py para o carrinho sumir.

O modo dev reinicia o servidor, e tudo o que estava na memória vai embora. Uma loja de verdade não pode perder o catálogo nem os pedidos a cada atualização.

CONCEITO

Persistência

Dado **persistente** é o que continua existindo depois que o programa termina. A memória (RAM) é rápida, mas **volátil**: quando o processo acaba, ela é devolvida ao sistema. Para persistir, o dado precisa ir para o **disco**, em geral por meio de um **banco de dados**.

exemplo

```
memória (RAM)              disco (banco)
─────────────              ─────────────
rápida                     mais lenta
some ao reiniciar          sobrevive ao reinício
só o processo enxerga      vários programas leem

CATALOGO = {...}   ->   tabela produto
```

PENSE ASSIM

A memória é o rascunho na mesa; o banco é o arquivo de aço da loja. O rascunho é rápido de usar, mas vai para o lixo no fim do dia.

FASE 1-1 · O QUE PRECISA SER GUARDADO?

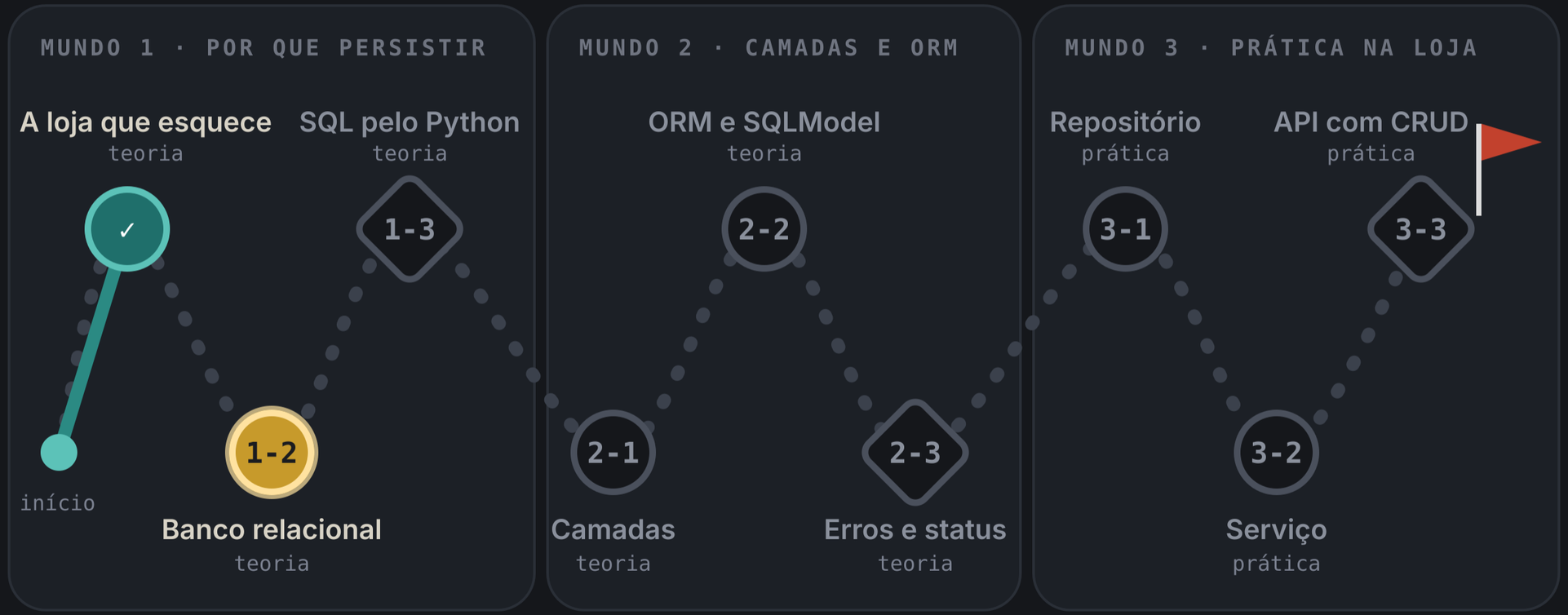
Na loja, o que deve ir para o banco?

- a) O catálogo de produtos.
- b) O total do carrinho, já calculado.
- c) Os pedidos finalizados.
- d) A promoção da semana.

a) e c): são fatos do negócio que não podem sumir.

b) é **derivado**: dá para recalcular a partir dos itens. Guardar o total é pedir para ele ficar desatualizado.

d) depende: se a dona muda toda semana sem programador, vai para o banco; se é regra fixa, fica no código.



FASE 1-2 · CONTEÚDO NOVO

Banco relacional

O modelo de tabelas que vocês já conhecem, agora visto do lado de quem programa.

CONCEITO

Banco de dados relacional

Organiza os dados em **tabelas**. Cada **linha** é um registro; cada **coluna**, um atributo. A **chave primária** identifica a linha sem repetição, e a restrição **UNIQUE** impede valores repetidos numa coluna.

exemplo

```
CREATE TABLE produto (
  id      INTEGER PRIMARY KEY,
  nome    TEXT NOT NULL UNIQUE,
  preco   REAL NOT NULL,
  tamanho TEXT NOT NULL
);

 id | nome             | preco  | tamanho
  1 | Camiseta básica  |  39.90 | M
  2 | Calça jeans      | 129.90 | G
```

CUIDADO

O banco também valida: `NOT NULL` e `UNIQUE` são uma segunda linha de defesa, além das regras das classes.

CONCEITO

SQLite

Um banco relacional que cabe num **arquivo**. Não precisa de servidor, instalação nem senha, e já vem com o Python. É o banco mais usado do mundo: está em celulares, navegadores e milhares de aplicativos.

exemplo

```
loja/
├── api.py
├── loja.db      <- o banco inteiro está aqui
└── loja/

# o mesmo SQL de sempre:
SELECT nome, preco FROM produto WHERE preco < 100;
```

PENSE ASSIM

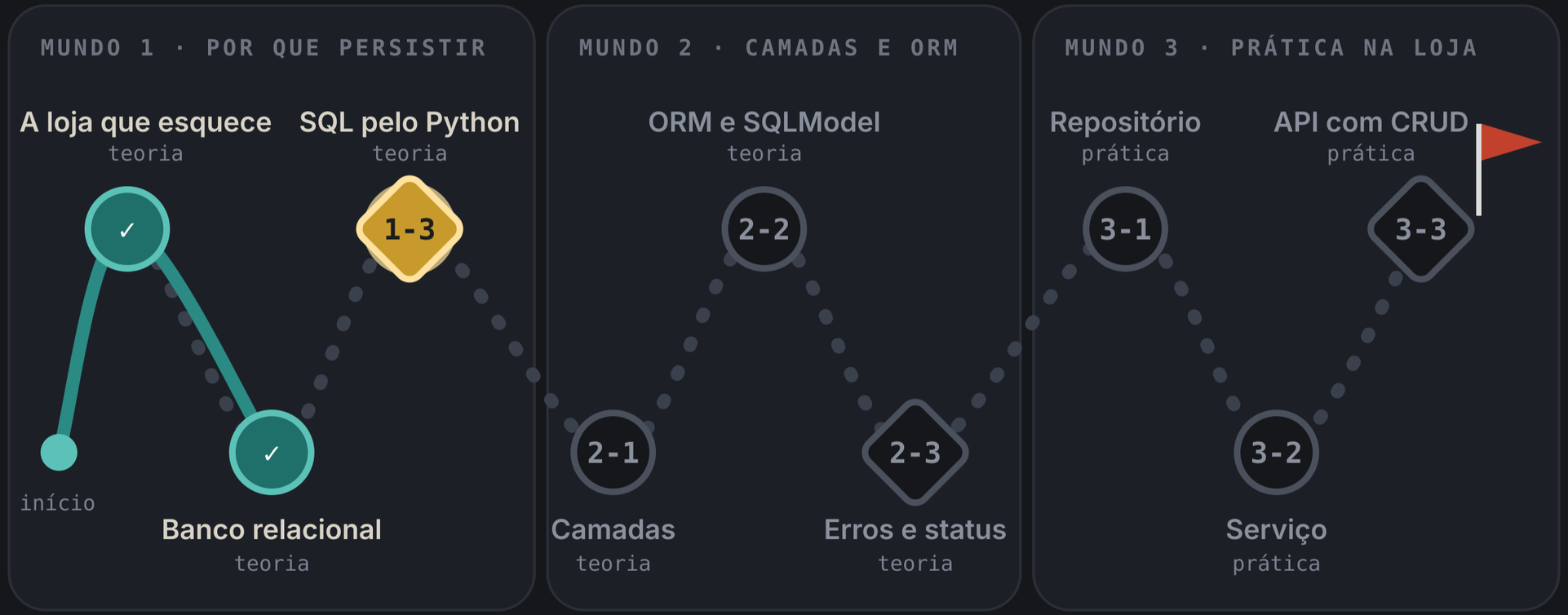
O SQLite é uma planilha guardada num arquivo; um servidor como o MySQL é um cartório com atendente, fila e senha.

FASE 1-2 · COMPARAÇÃO

SQLite ou MySQL: quando usar cada um

	SQLITE	MYSQL
Onde mora	um arquivo na pasta do projeto	um servidor, com porta, usuário e senha
Instalação	nenhuma: vem com o Python	instalar e configurar o servidor
Vários acessos ao mesmo tempo	leituras sim; escritas uma de cada vez	muitos usuários escrevendo juntos
Bom para	aprender, testar, apps locais e pequenos	sistemas com muitos usuários simultâneos
Nos testes automáticos	banco em memória, em milissegundos	precisa do servidor no ar

Na loja e no projeto: SQLite. Na Fase 2-2, vocês vão ver que trocar para MySQL é trocar uma linha.



FASE 1-3 · CONTEÚDO NOVO

SQL pelo Python

Antes de qualquer biblioteca: o SQL que vocês conhecem, rodando de dentro do programa.

CONCEITO

Conexão, cursor e commit

A **conexão** abre o banco. O **cursor** executa os comandos SQL e percorre os resultados. O **commit** confirma as alterações: sem ele, nada do que foi inserido ou alterado fica gravado.

exemplo

```
conexao = sqlite3.connect("exemplo.db")
cursor  = conexao.cursor()

cursor.execute("INSERT ...")
conexao.commit()      # agora está no disco

for linha in cursor.execute("SELECT ..."):
    print(linha)      # cada linha é uma tupla
```

PENSE ASSIM

Escrever a lápis e passar a caneta: o execute escreve a lápis, o commit passa a caneta.

FASE 1-3 · EXEMPLO

exemplos/sqlite_puro.py: o SQL de sempre, dentro do Python

exemplos/sqlite_puro.py

```python
import sqlite3

conexao = sqlite3.connect("exemplo.db")
cursor = conexao.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS produto (
        id      INTEGER PRIMARY KEY AUTOINCREMENT,
        nome    TEXT NOT NULL UNIQUE,
        preco   REAL NOT NULL,
        tamanho TEXT NOT NULL
    )
""")

# o ? protege contra SQL injection: o valor nunca é colado no texto do SQL
cursor.execute("INSERT OR IGNORE INTO produto (nome, preco, tamanho) VALUES (?, ?, ?)",
               ("Camiseta básica", 39.90, "M"))
conexao.commit()

for linha in cursor.execute("SELECT id, nome, preco FROM produto WHERE preco < ?", (100,)):
    print(linha)

conexao.close()
```

- `connect` cria o arquivo se ele não existir.
- Os `?` são **parâmetros**: o valor vai separado do texto do SQL.
- Sem `commit()`, o INSERT se perde quando o programa termina.
- Cada linha do SELECT chega como **tupla**: `(1, 'Camiseta básica', 39.9)`.

CONCEITO

SQL injection

Ataque em que um texto digitado pelo usuário **vira parte do comando SQL**. Acontece quando o programa monta o SQL colando strings. Com **parâmetros** (`?`), o banco trata o valor sempre como dado, nunca como comando.

exemplo

```
nome = "x' OR '1'='1"

# PERIGO: o texto vira SQL
f"SELECT * FROM produto WHERE nome = '{nome}'"
# -> devolve TODOS os produtos

# CERTO: o valor vai separado
cursor.execute("SELECT * FROM produto WHERE nome = ?", (nome,))
```

CUIDADO

Nunca use f-string para montar SQL com dado vindo de fora. O ORM da Fase 2-2 usa parâmetros automaticamente.

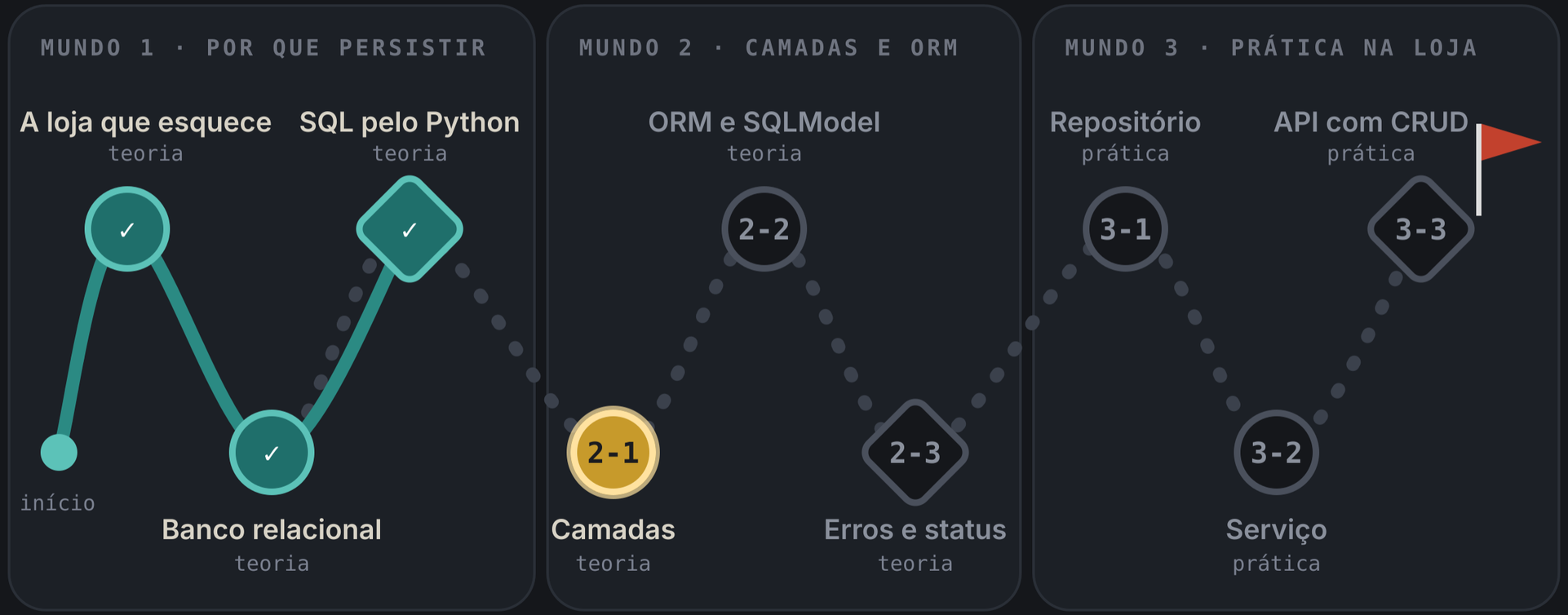
FASE 1-3 · DIAGNÓSTICO

O programa roda sem erro, mas o produto não aparece no banco. Por quê?

cadastro.py

```
conexao = sqlite3.connect("loja.db")
cursor = conexao.cursor()
cursor.execute(
    "INSERT INTO produto (nome, preco, tamanho) VALUES (?, ?, ?)",
    ("Boné", 49.90, "M"))
conexao.close()
```

Faltou o **commit()** antes do `close()`. O INSERT ficou pendente e foi descartado quando a conexão fechou.



FASE 2-1 · CONTEÚDO NOVO

Camadas

Se a rota fizer tudo (HTTP, regra e SQL), qualquer mudança quebra tudo. A saída é dividir o trabalho.

CONCEITO

Arquitetura em camadas

O código é dividido em **camadas**, cada uma com **uma responsabilidade**. Uma camada só conversa com a de baixo, e não sabe como a de baixo faz o trabalho. Mudar uma camada não obriga a mexer nas outras.

exemplo

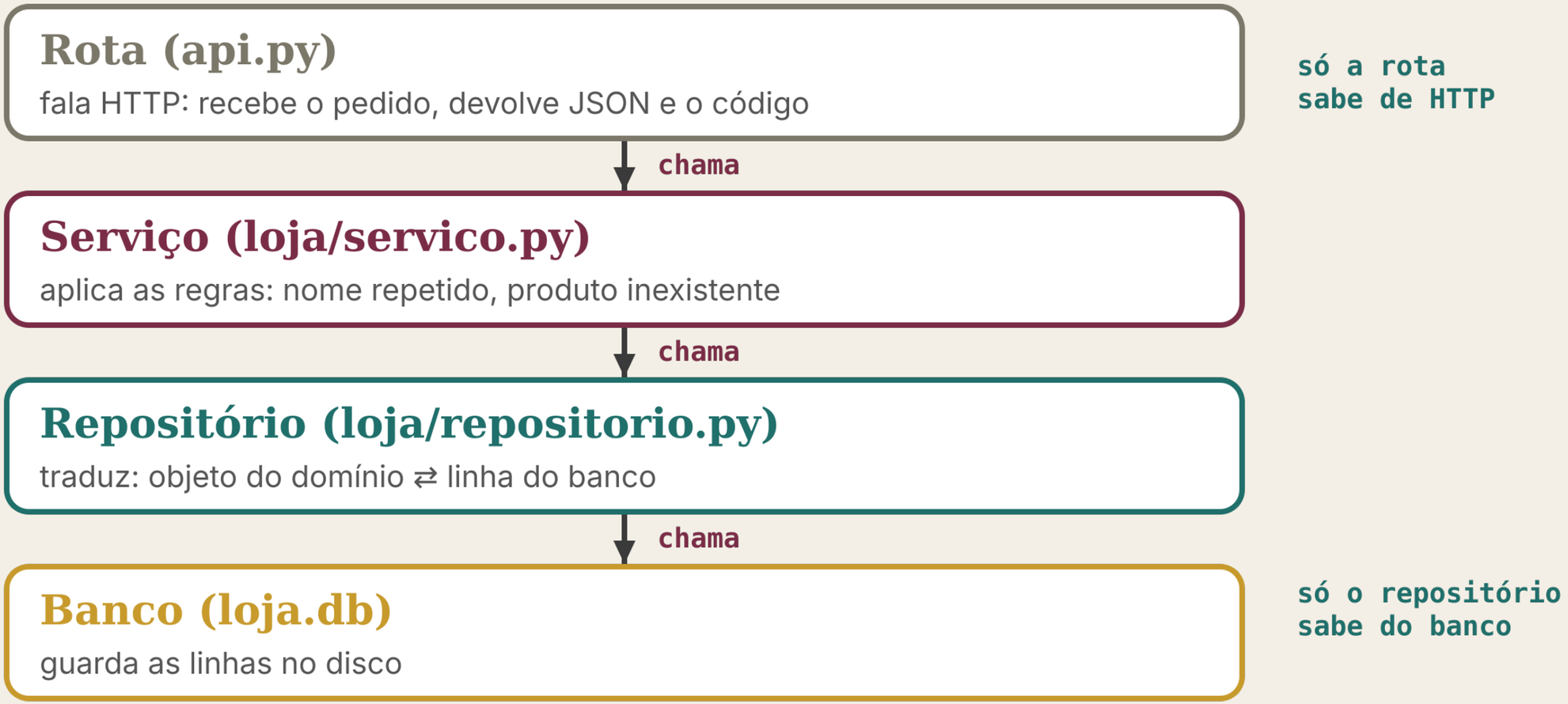
```
rota         fala HTTP
  |
serviço      aplica as regras do negócio
  |
repositório  traduz objeto <-> linha
  |
banco        guarda no disco
```

PENSE ASSIM

O restaurante: o garçom anota (rota), o chef decide como fazer (serviço), o estoquista busca na despensa (repositório). O cliente nunca entra na despensa.

FASE 2-1 · NA IMAGEM

Quem chama quem na Loja de Roupas



FASE 2-1 · COMPARAÇÃO

O que cada camada sabe, e o que não sabe

CAMADA	SABE	NÃO SABE
Rota (`api.py`)	métodos HTTP, JSON, códigos de status	regras do negócio, SQL
Serviço (`servico.py`)	regras: nome repetido, produto inexistente	HTTP e qual banco existe
Repositório (`repositorio.py`)	como gravar e buscar no banco	HTTP e regras do negócio
Domínio (`produto.py`, ...)	invariantes: preço positivo, tamanho válido	HTTP, banco, tudo de fora

Teste rápido: se a frase "o serviço sabe que existe um 404" for verdadeira no seu código, a separação vazou.

CONCEITO

Repositório

Uma classe que **esconde o banco** atrás de métodos com nomes do negócio: `adicionar`, `buscar`, `listar`, `atualizar`, `remover`. Quem usa o repositório recebe e entrega **objetos do domínio**, nunca linhas de tabela.

exemplo

```
repo = RepositorioProdutos(sessao)

produto_id = repo.adicionar(camiseta)
produto    = repo.buscar(produto_id)   # um Camiseta
repo.listar(tamanho="M")
repo.remover(produto_id)
```

PENSE ASSIM

O estoquista: você pede "a camiseta 7" e recebe a camiseta. Em qual prateleira ela estava não é problema seu.

CONCEITO

Serviço

A camada das **regras de negócio** que envolvem mais de um objeto ou consultam o que já existe. Ele usa o repositório e, quando algo dá errado, levanta uma **exceção com nome do negócio**.

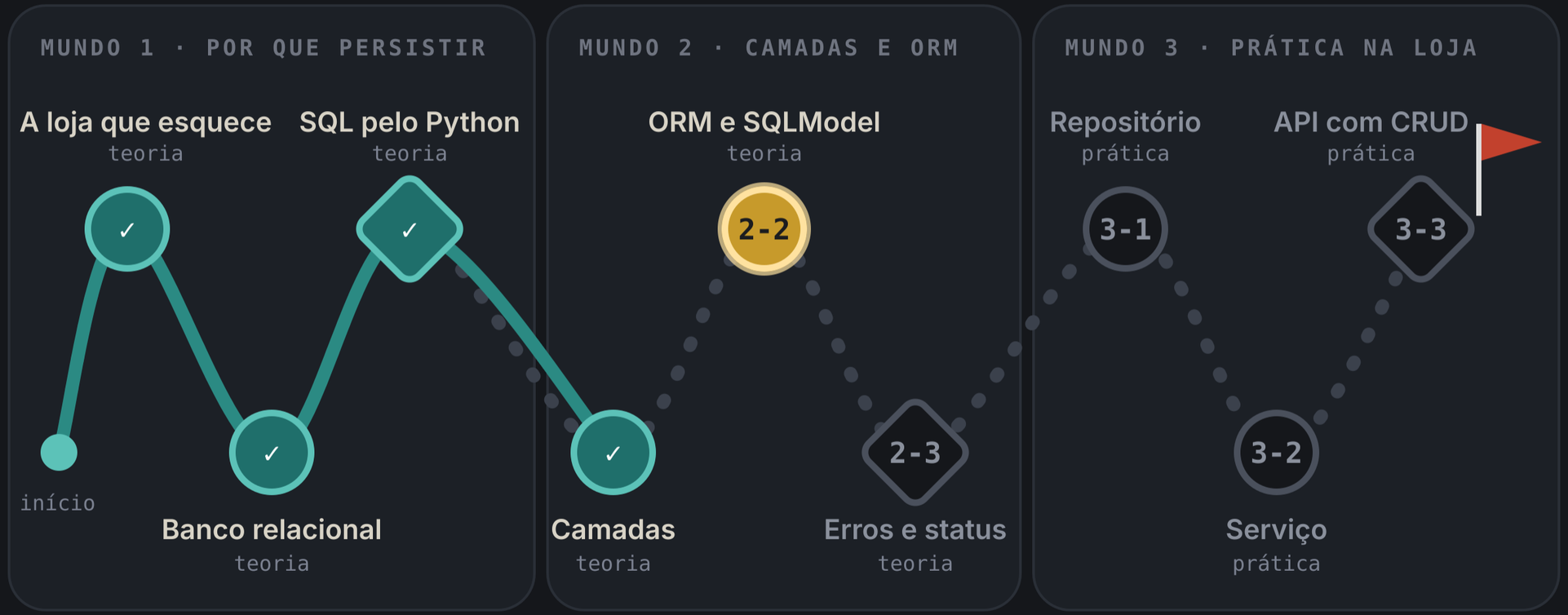
exemplo

```
servico = ServicoCatalogo(repo)

servico.cadastrar(dados)
# nome já existe?  -> ProdutoJaExiste
# preço zero?       -> DadosInvalidos
# tudo certo        -> devolve com id
```

CUIDADO

Regra que vale para um único objeto (preço positivo) fica na classe do domínio. Regra que precisa olhar o banco (nome único) fica no serviço.



FASE 2-2 · CONTEÚDO NOVO

ORM e SQLModel

Escrever SQL à mão para cada consulta cansa e erra. Um ORM faz a tradução entre classes e tabelas.

CONCEITO

ORM (mapeamento objeto-relacional)

Uma biblioteca que **traduz** entre o mundo dos objetos e o mundo das tabelas. A classe vira tabela, o objeto vira linha, o atributo vira coluna, e o SQL é gerado pelo ORM, sempre com parâmetros.

exemplo

Python	banco
classe ProdutoRegistro	tabela produto
objeto	linha
atributo preco	coluna preco
sessao.add(obj)	INSERT INTO ...
sessao.get(Cls, 1)	SELECT ... WHERE id = ?

PENSE ASSIM

Um intérprete numa reunião: cada um fala a própria língua, e o intérprete traduz nos dois sentidos.

CONCEITO

SQLModel

O ORM do mesmo autor do FastAPI. Uma classe com `table=True` vira **tabela**; `Field` define chave primária e restrições. Por baixo, ele usa o SQLAlchemy, o ORM mais usado do Python.

exemplo

```
class ProdutoRegistro(SQLModel, table=True):
    id: int | None = Field(default=None,
                           primary_key=True)
    nome: str = Field(unique=True)
    preco: float
    tamanho: str
```

CUIDADO

O tipo `int | None` no id é porque, antes de gravar, o produto ainda não tem id: quem gera é o banco.

CONCEITO

Sessão

A **conversa** com o banco. Você acumula mudanças com `add` e `delete`, e o `commit` grava tudo de uma vez. `get` busca pela chave primária; `exec(select(...))` faz consultas com filtro.

exemplo

```
with Session(engine) as sessao:
    sessao.add(registro)
    sessao.commit()          # grava
    sessao.refresh(registro) # pega o id gerado

    sessao.get(ProdutoRegistro, 1)
    sessao.exec(select(ProdutoRegistro)
        .where(ProdutoRegistro.preco <= 100))
```

FASE 2-2 · CONCEITO

Por que duas classes: Produto e ProdutoRegistro

LOJA/PRODUTO.PY · DOMÍNIO

Produto, Camiseta, Calca

As classes das Aulas 6 a 8. Validam invariantes e têm comportamento (`descricao`, herança, polimorfismo). **Não sabem que existe banco.**

LOJA/BANCO.PY · TABELA

ProdutoRegistro

Uma classe "plana", do jeito que a tabela guarda: uma coluna `tipo` diz se é Camiseta ou Calca. O **repositório** traduz entre as duas.

Assim o domínio continua testável sem banco, e os 30 testes das Aulas 6 a 8 não mudam uma linha.

FASE 2-2 · PONTE

SQLite hoje, MySQL amanhã: muda uma linha

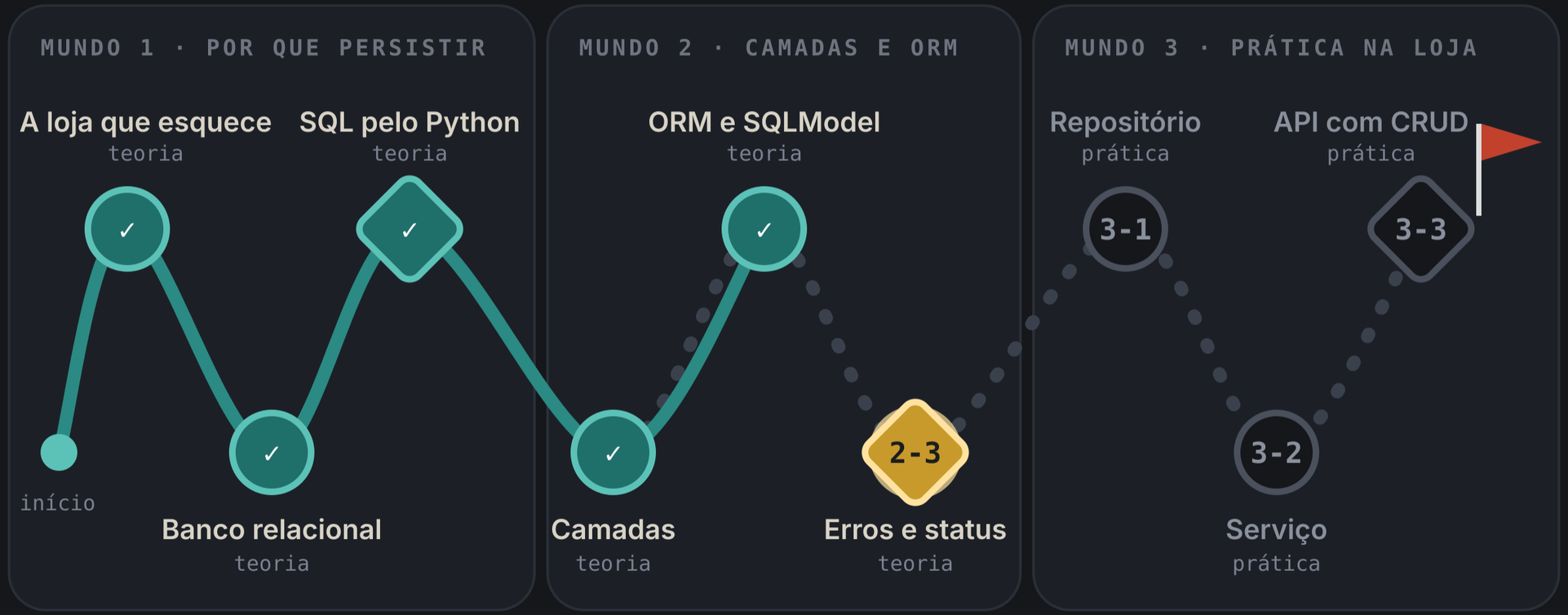
loja/banco.py

```python
URL_BANCO = "sqlite:///loja.db"

# trocar para o MySQL do laboratório:
# pip install pymysql cryptography
# URL_BANCO = "mysql+pymysql://usuario:senha@localhost:3306/loja"


def criar_engine(url=URL_BANCO):
    argumentos = {"check_same_thread": False} if url.startswith("sqlite") else {}
    return create_engine(url, connect_args=argumentos)
```

- Rota, serviço e repositório **continuam iguais**: só a URL muda.
- No MySQL, o banco `loja` precisa existir antes (`CREATE DATABASE loja;`).
- `check_same_thread` é um argumento só do SQLite: por isso o `if`, e o MySQL não recebe o que não conhece.
- A senha não vai para o GitHub: fica num arquivo `.env`, fora do repositório.



FASE 2-3 · CONTEÚDO NOVO

Erros e status

O serviço fala a língua do negócio. A rota traduz para a língua do HTTP.

CONCEITO

Exceção do domínio

Uma classe de exceção com **nome do negócio**, levantada pelo serviço quando uma regra não é cumprida. Quem decide qual **código HTTP** ela vira é a camada de cima, num único lugar.

exemplo

```
class ProdutoNaoEncontrado(Exception): ...
class ProdutoJaExiste(Exception): ...
class DadosInvalidos(Exception): ...

raise ProdutoJaExiste("já existe um produto chamado Boné")
```

PENSE ASSIM

O chef diz "acabou o salmão". É o garçom que traduz isso para o cliente, do jeito da casa.

FASE 2-3 · TABELA

Cada exceção vira um código HTTP

SITUAÇÃO	QUEM PERCEBE	EXCEÇÃO	CÓDIGO
JSON sem o campo `preco`	Pydantic, na rota	(automática)	**422**
preço zero, tamanho XL	classe do domínio	`DadosInvalidos`	**422** Unprocessable Content
produto 99 não existe	serviço	`ProdutoNaoEncontrado`	**404** Not Found
já existe um "Boné"	serviço	`ProdutoJaExiste`	**409** Conflict
cadastro deu certo	rota	(nenhuma)	**201** Created
remoção deu certo	rota	(nenhuma)	**204** No Content

FASE 2-3 · FLUXOGRAMA

O caminho de um POST /produtos



CONCEITO

Injeção de dependência (Depends)

Em vez de a rota **criar** a sessão e o serviço, ela **declara** que precisa deles, e o FastAPI entrega pronto a cada pedido. Nos testes, dá para **trocar** a dependência por um banco em memória.

exemplo

```
def obter_servico(sessao = Depends(obter_sessao)):
    return ServicoCatalogo(RepositorioProdutos(sessao))

@app.get("/produtos/{produto_id}")
def buscar(produto_id: int,
           servico = Depends(obter_servico)):
    return servico.buscar(produto_id)
```

CUIDADO

Uma sessão por pedido HTTP: abre no começo, fecha no fim. Nunca uma sessão global para o servidor inteiro.

FASE 2-3 · DIAGNÓSTICO

Qual código a API deve responder?

- a) `PUT /produtos/7`, e não existe produto 7.
- b) `POST /produtos` com o nome de um produto que já existe.
- c) `POST /produtos` com tamanho "XL".
- d) `DELETE /produtos/2`, que existe.

a) 404: o recurso não existe.

b) 409: conflito com o que já está gravado.

c) 422: o formato está certo, mas a regra do domínio recusa.

d) 204: removido, nada para devolver.

HORA DA PRÁTICA

Agora é com vocês

1

Abrir esta aula

no site, em Materiais › Coding, na Fase 3-1 (tecla M abre o mapa)

2

Partir da etapa-12

o projeto da loja como ficou na Aula 9, ou baixar a tag pronta

3

Seguir as três fases

repositório, serviço e API com CRUD

4

Rodar o pytest a cada fase

cada checkpoint diz quantos testes devem passar

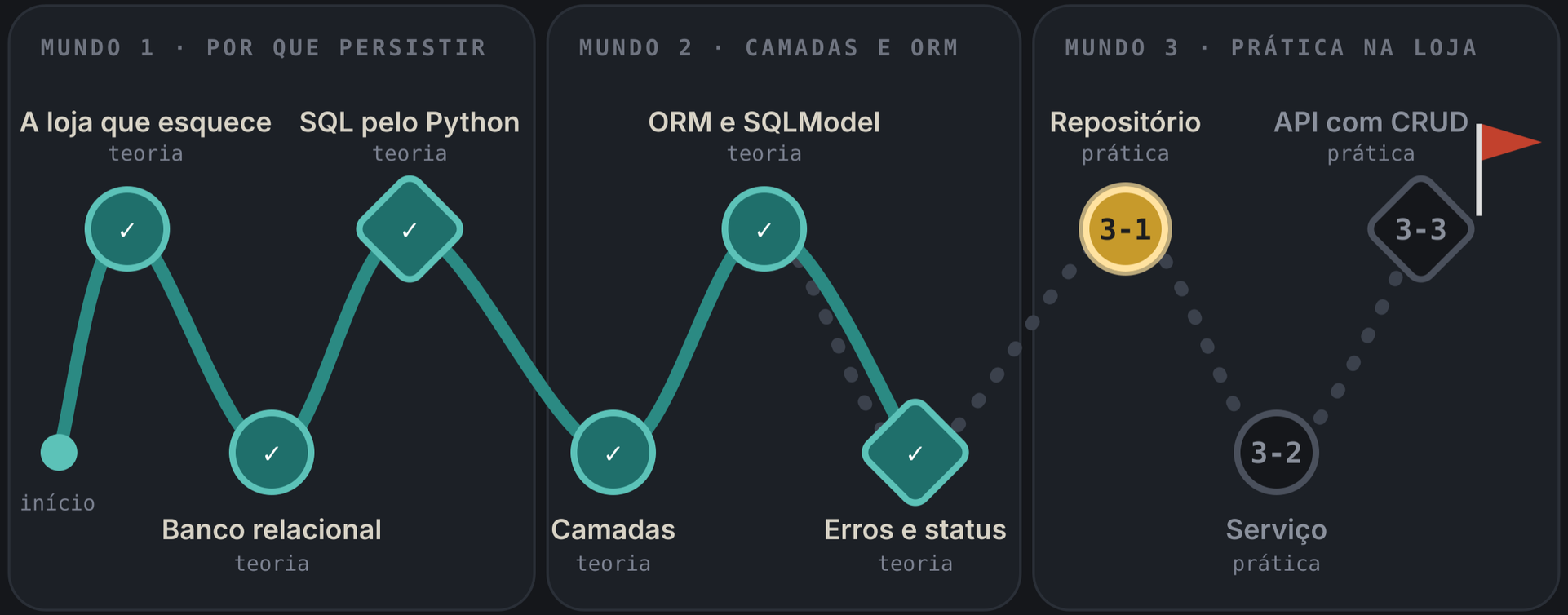
5

Travou?

o checkpoint tem o link da fase pronta; depois, chame o professor

EM DUPLA

Um digita, o outro lê o slide e confere. Troquem a cada fase.



FASE 3-1 · PRÁTICA

Repositório

A tabela produto, a conexão com o SQLite e o repositório com CRUD, testado com banco em memória.

FASE 3-1 · NO TERMINAL

Instalar o SQLModel

terminal

```
$ pip install sqlmodel
Successfully installed sqlmodel ... sqlalchemy ...
$ pip freeze | grep -i sqlmodel
sqlmodel==0.0.x
```

- Com o `.venv` ativo, como sempre.
- Acrescente `sqlmodel` ao `requirements.txt`: o GitHub Actions instala por ele.
- No `.gitignore`, acrescente `loja.db`: o banco local não vai para o repositório.

FASE 3-1 · NO EDITOR

loja/banco.py: a tabela

loja/banco.py

```python
from sqlalchemy.pool import StaticPool
from sqlmodel import Field, SQLModel, create_engine

URL_BANCO = "sqlite:///loja.db"


class ProdutoRegistro(SQLModel, table=True):
    """Uma linha da tabela produto: o produto do jeito que o banco guarda."""
    __tablename__ = "produto"

    id: int | None = Field(default=None, primary_key=True)
    tipo: str
    nome: str = Field(unique=True)
    preco: float
    tamanho: str
    manga: str | None = None
    modelagem: str | None = None
```

- Uma tabela só, com uma coluna `tipo` para Produto, Camiseta e Calca.
- `primary_key` é o id que o banco gera; `unique` impede dois produtos com o mesmo nome.
- A URL do banco fica em um lugar só.

FASE 3-1 · NO EDITOR

loja/banco.py: a conexão

loja/banco.py (continuação)

```python
def criar_engine(url=URL_BANCO):
    # check_same_thread é um argumento só do SQLite: outros bancos não o conhecem
    argumentos = {"check_same_thread": False} if url.startswith("sqlite") else {}
    return create_engine(url, connect_args=argumentos)


def engine_em_memoria():
    """Banco que vive só na memória: ideal para testes rápidos."""
    return create_engine("sqlite://", connect_args={"check_same_thread": False},
                         poolclass=StaticPool)


def criar_tabelas(engine):
    SQLModel.metadata.create_all(engine)
```

- `criar_engine` só passa `check_same_thread` ao SQLite: outros bancos não conhecem esse argumento.
- `engine_em_memoria` cria um banco que só existe enquanto o teste roda.
- `create_all` cria as tabelas que ainda não existem, e não apaga dados.

FASE 3-1 · NO EDITOR

loja/repositorio.py: traduzir nos dois sentidos

loja/repositorio.py

```python
from sqlmodel import Session, select

from loja.banco import ProdutoRegistro
from loja.produto import Calca, Camiseta, Produto

CAMPOS = ("tipo", "nome", "preco", "tamanho", "manga", "modelagem")


def para_dominio(registro):
    """Linha do banco -> objeto do domínio (com as validações de sempre)."""
    if registro.tipo == "Camiseta":
        return Camiseta(registro.nome, registro.preco, registro.tamanho, registro.manga)
    if registro.tipo == "Calca":
        return Calca(registro.nome, registro.preco, registro.tamanho, registro.modelagem)
    return Produto(registro.nome, registro.preco, registro.tamanho)

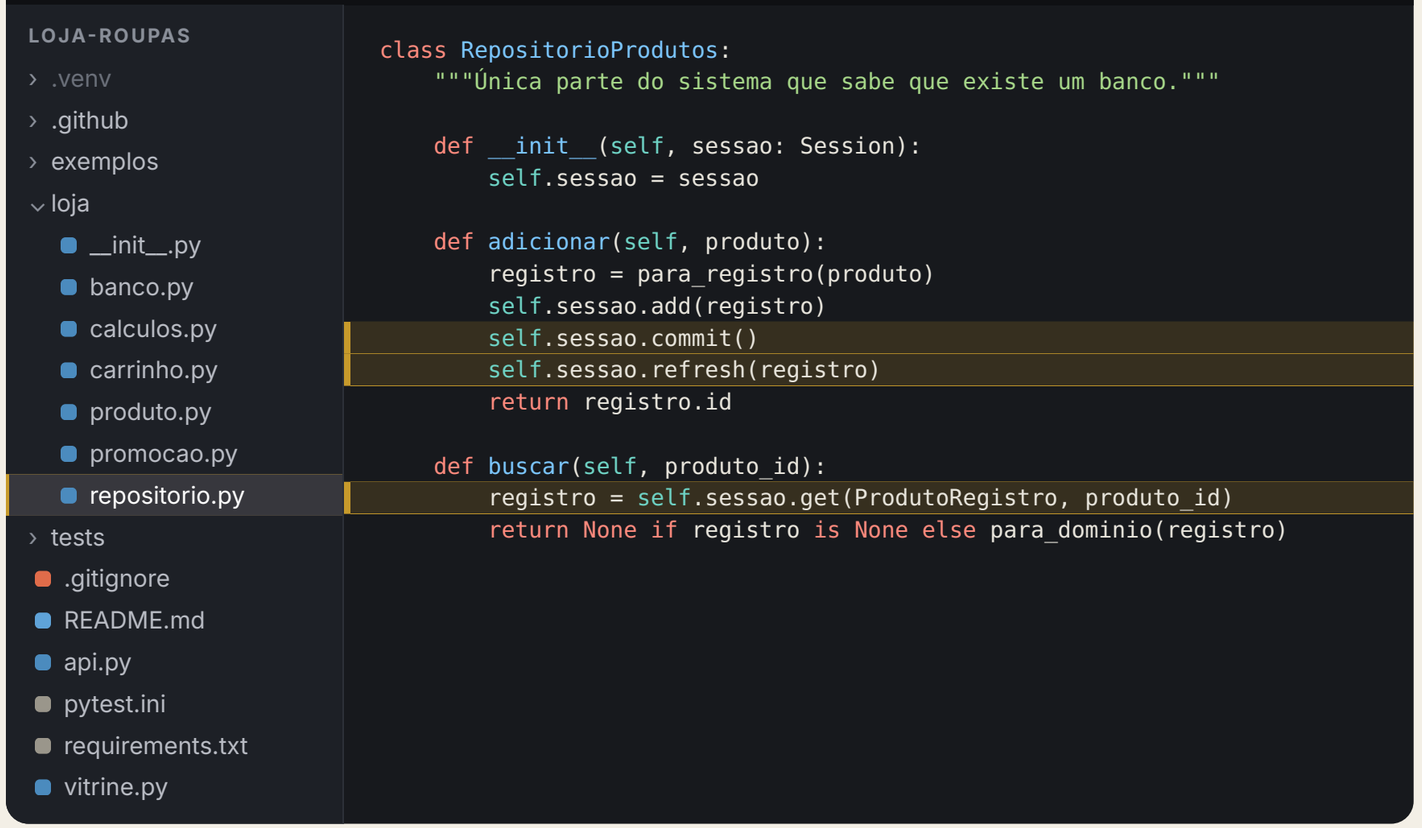

def para_registro(produto):
    """Objeto do domínio -> linha do banco."""
    return ProdutoRegistro(**produto.para_dict())
```

- `para_dominio` reconstrói o objeto certo pela coluna `tipo`, e as validações do construtor rodam de novo.
- `para_registro` reaproveita o `para_dict` da Aula 9.

FASE 3-1 · NO EDITOR

loja/repositorio.py: gravar e buscar

loja/repositorio.py (continuação)



- `commit` grava; `refresh` traz de volta o id que o banco gerou.
- `get` busca pela chave primária e devolve `None` se não achar.

FASE 3-1 · NO EDITOR

loja/repositorio.py: consultar com filtro

loja/repositorio.py (continuação)

```
    def existe_nome(self, nome, exceto_id=None):
        consulta = select(ProdutoRegistro).where(ProdutoRegistro.nome == nome)
        registro = self.sessao.exec(consulta).first()
        return registro is not None and registro.id != exceto_id

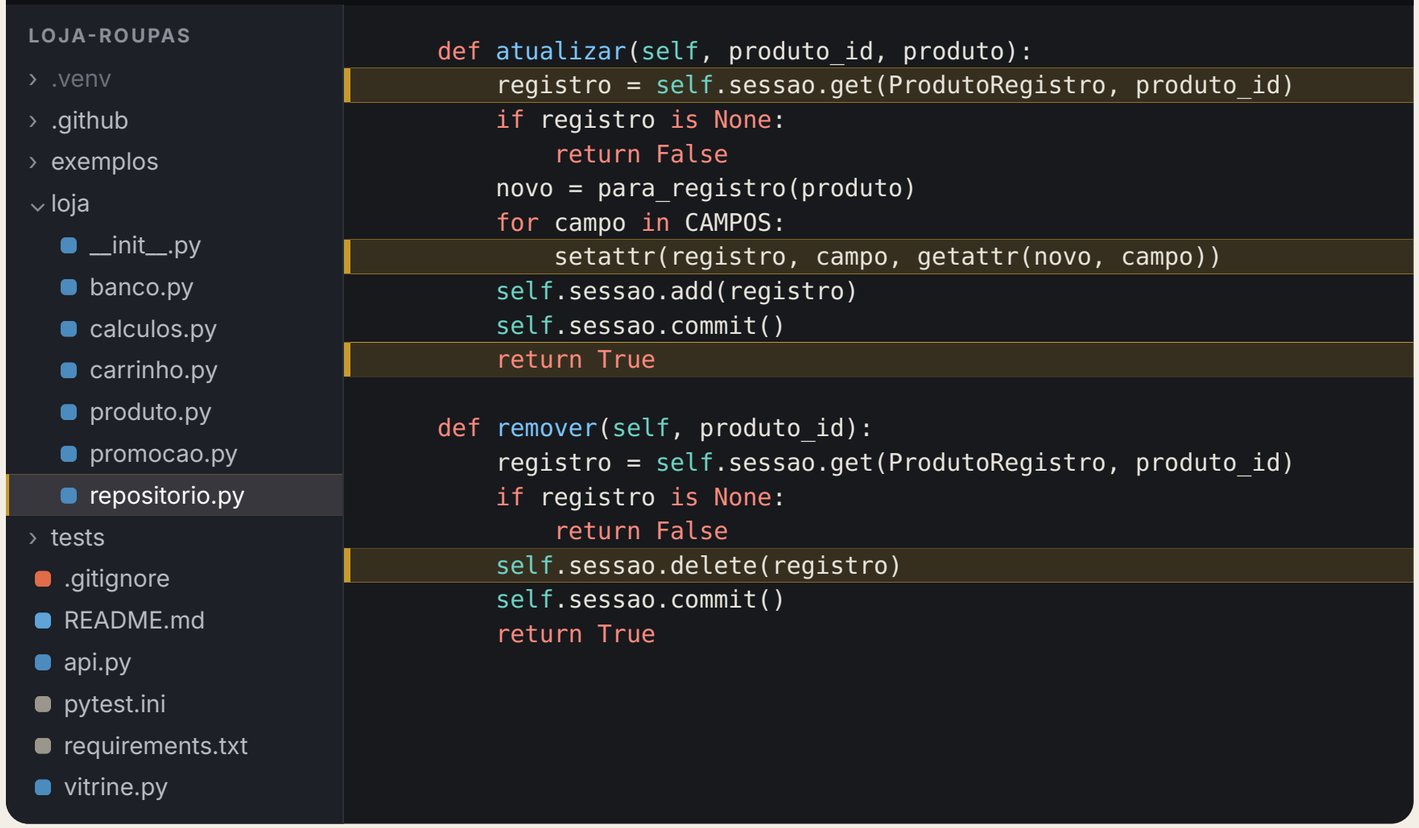
    def listar(self, tamanho=None, preco_max=None):
        consulta = select(ProdutoRegistro)
        if tamanho is not None:
            consulta = consulta.where(ProdutoRegistro.tamanho == tamanho)
        if preco_max is not None:
            consulta = consulta.where(ProdutoRegistro.preco <= preco_max)
        consulta = consulta.order_by(ProdutoRegistro.id)
        return [(r.id, para_dominio(r)) for r in self.sessao.exec(consulta)]
```

- `select(...).where(...)` é o SELECT com WHERE, escrito em Python e com parâmetros.
- `listar` monta a consulta aos poucos: só filtra o que foi pedido.
- Quem chama sabe só o que o repositório devolve: objetos do domínio.

FASE 3-1 · NO EDITOR

loja/repositorio.py: atualizar e remover

loja/repositorio.py (continuação)




```
    def atualizar(self, produto_id, produto):
        registro = self.sessao.get(ProdutoRegistro, produto_id)
        if registro is None:
            return False
        novo = para_registro(produto)
        for campo in CAMPOS:
            setattr(registro, campo, getattr(novo, campo))
        self.sessao.add(registro)
        self.sessao.commit()
        return True

    def remover(self, produto_id):
        registro = self.sessao.get(ProdutoRegistro, produto_id)
        if registro is None:
            return False
        self.sessao.delete(registro)
        self.sessao.commit()
        return True
```

- Os dois seguem o mesmo jeito: `get`, mudar ou apagar, `commit`.
- `atualizar` copia campo a campo o produto novo para a linha do banco.
- Devolvem `False` se o id não existe: quem decide o que fazer com isso é o serviço.

FASE 3-1 · NO EDITOR

tests/test_repositorio.py: a prova de que não esquece

tests/test_repositorio.py

```python
def test_dados_sobrevivem_ao_reinicio(tmp_path):
    url = f"sqlite:///{tmp_path / 'loja.db'}"
    engine = criar_engine(url)
    criar_tabelas(engine)
    with Session(engine) as sessao:
        RepositorioProdutos(sessao).adicionar(Produto("Moletom", 159.90, "P"))
    engine.dispose()

    outro = criar_engine(url)
    with Session(outro) as sessao:
        assert RepositorioProdutos(sessao).buscar(1).nome == "Moletom"
```

- `tmp_path` é uma pasta temporária que o pytest cria e apaga.
- Grava, "desliga" o banco e abre de novo: o Moletom continua lá.
- Os outros testes do arquivo usam o banco em memória.

FASE 3-1 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
46 passed
$ git add .
$ git commit -m "Fase 3-1: banco SQLite e repositório"
$ git push
```

CONFIRA

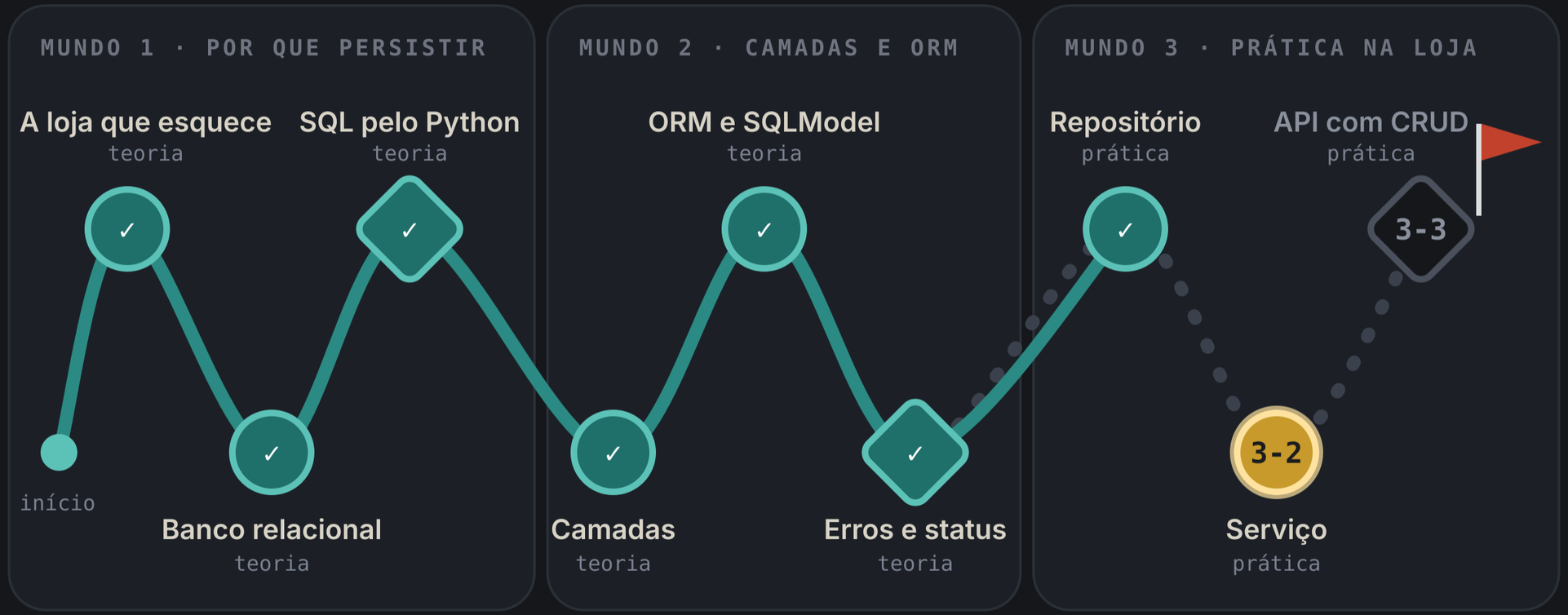
O `pytest` mostra **46 passed**: os 40 de antes e 6 do repositório.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-13.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-13`



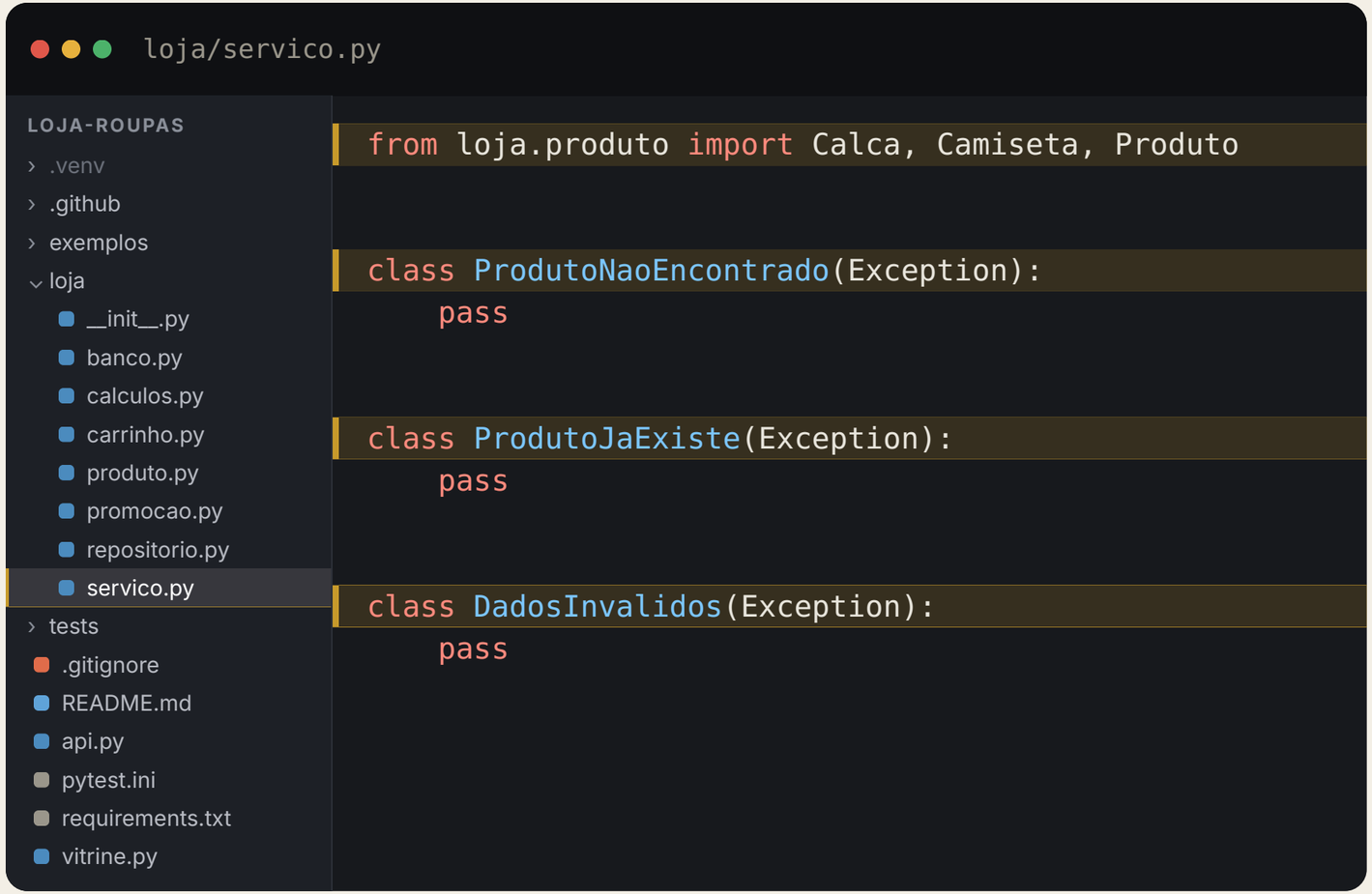
FASE 3-2 · PRÁTICA

Serviço

As regras do catálogo e as exceções com nome de negócio, sem nenhuma linha de HTTP.

FASE 3-2 · NO EDITOR

loja/servico.py: imports e exceções



- O serviço importa só o **domínio**: nenhuma linha de banco ou de HTTP.
- Cada exceção tem **nome de negócio**; quem decide o código HTTP é a rota.

FASE 3-2 · NO EDITOR

loja/servico.py: criar o produto

loja/servico.py (continuação)

```
def criar_produto(dados):
    """Dicionário -> objeto do domínio. Erros de validação viram DadosInvalidos."""
    try:
        tipo = dados.get("tipo", "Produto")
        if tipo == "Camiseta":
            return Camiseta(dados["nome"], dados["preco"], dados["tamanho"], dados.get("manga"))
        if tipo == "Calca":
            if not dados.get("modelagem"):
                raise ValueError("calça precisa de modelagem")
            return Calca(dados["nome"], dados["preco"], dados["tamanho"], dados["modelagem"])
        if tipo == "Produto":
            return Produto(dados["nome"], dados["preco"], dados["tamanho"])
        raise ValueError(f"tipo inválido: {tipo}")
    except (ValueError, KeyError) as erro:
        raise DadosInvalidos(str(erro)) from erro
```

- O `ValueError` das classes do domínio vira `DadosInvalidos`.
- `from erro` guarda a causa original, para quem for depurar.
- Tipo desconhecido ("Sapato") também é dado inválido.

FASE 3-2 · NO EDITOR

loja/servico.py: o ServicoCatalogo

loja/servico.py (continuação)

```python
def com_id(produto_id, produto):
    return {"id": produto_id, **produto.para_dict()}


class ServicoCatalogo:
    """Recebe o repositório pronto: não sabe se o banco é SQLite, MySQL ou memória."""

    def __init__(self, repositorio):
        self.repositorio = repositorio

    def listar(self, tamanho=None, preco_max=None):
        return [com_id(i, p) for i, p in self.repositorio.listar(tamanho, preco_max)]

    def produto(self, produto_id):
        produto = self.repositorio.buscar(produto_id)
        if produto is None:
            raise ProdutoNaoEncontrado(f"produto {produto_id} não encontrado")
        return produto

    def buscar(self, produto_id):
        return com_id(produto_id, self.produto(produto_id))

    def cadastrar(self, dados):
        produto = criar_produto(dados)
        if self.repositorio.existe_nome(produto.nome):
            raise ProdutoJaExiste(f"já existe um produto chamado {produto.nome}")
        return com_id(self.repositorio.adicionar(produto), produto)
```

- O serviço **recebe** o repositório pronto: não sabe se o banco é SQLite, MySQL ou memória.
- Nome único é regra do serviço porque precisa consultar o que já está gravado.
- `com_id` junta o id ao dicionário do produto: é o que a API devolve.
- Nenhum `404` aqui: o serviço não sabe que existe HTTP.

FASE 3-2 · NO EDITOR

loja/servico.py: atualizar e remover

loja/servico.py (continuação)

```
    def atualizar(self, produto_id, dados):
        self.produto(produto_id)
        produto = criar_produto(dados)
        if self.repositorio.existe_nome(produto.nome, exceto_id=produto_id):
            raise ProdutoJaExiste(f"já existe um produto chamado {produto.nome}")
        self.repositorio.atualizar(produto_id, produto)
        return com_id(produto_id, produto)

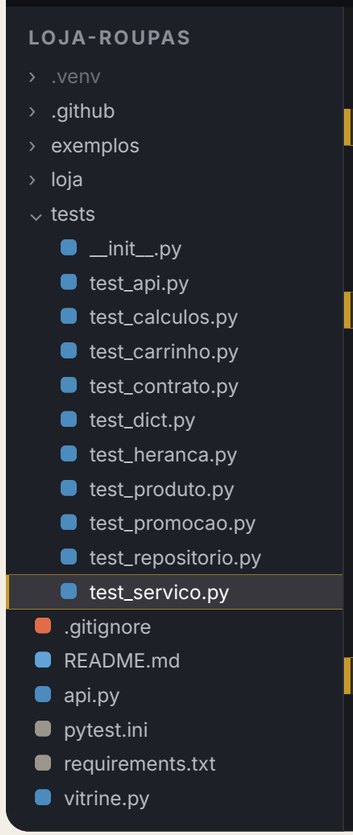
    def remover(self, produto_id):
        if not self.repositorio.remover(produto_id):
            raise ProdutoNaoEncontrado(f"produto {produto_id} não encontrado")
```

- `atualizar` primeiro confirma que o produto existe: se não, `ProdutoNaoEncontrado`.
- `exceto_id` deixa o produto manter o próprio nome sem dar conflito.
- `remover` transforma o `False` do repositório em exceção do negócio.

FASE 3-2 · NO EDITOR

tests/test_servico.py: as regras, testadas

tests/test_servico.py




```python
def test_nome_repetido_nao_entra(servico):
    servico.cadastrar(CAMISETA)
    with pytest.raises(ProdutoJaExiste):
        servico.cadastrar(CAMISETA)


def test_regra_do_dominio_vira_dados_invalidos(servico):
    with pytest.raises(DadosInvalidos):
        servico.cadastrar({**CAMISETA, "preco": 0})


def test_tipo_desconhecido_e_invalido(servico):
    with pytest.raises(DadosInvalidos):
        servico.cadastrar({**CAMISETA, "tipo": "Sapato"})


def test_buscar_inexistente(servico):
    with pytest.raises(ProdutoNaoEncontrado):
        servico.buscar(99)
```

- `pytest.raises` confere que a exceção certa foi levantada.
- Cada teste ganha um banco novo, em memória, pela fixture.

FASE 3-2 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
53 passed
$ git add .
$ git commit -m "Fase 3-2: serviço do catálogo"
$ git push
```

CONFIRA

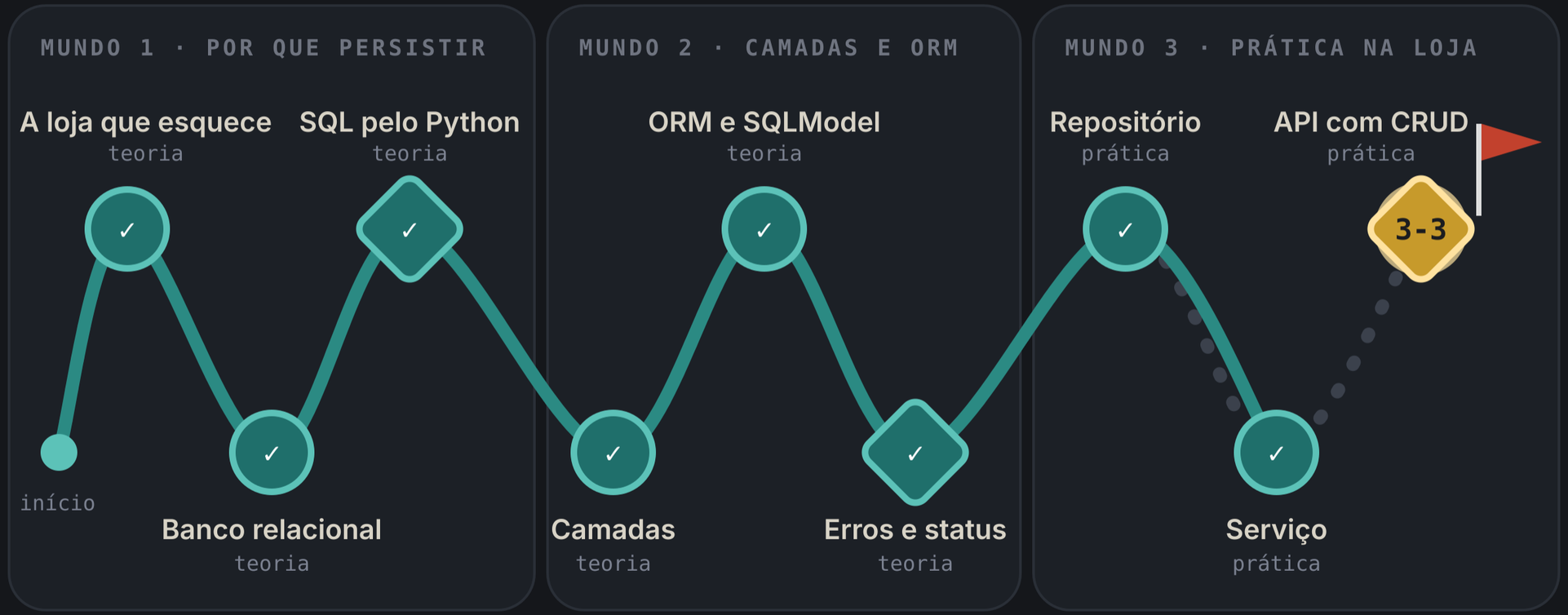
O `pytest` mostra **53 passed**: mais 7 testes do serviço.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-14.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-14`



FASE 3-3 · PRÁTICA

API com CRUD

A rota passa a usar o serviço. Cadastrar, listar, alterar e remover, tudo gravado no loja.db.

FASE 3-3 · NO EDITOR

api.py: o que a rota importa

api.py

```python
from contextlib import asynccontextmanager
from typing import Literal

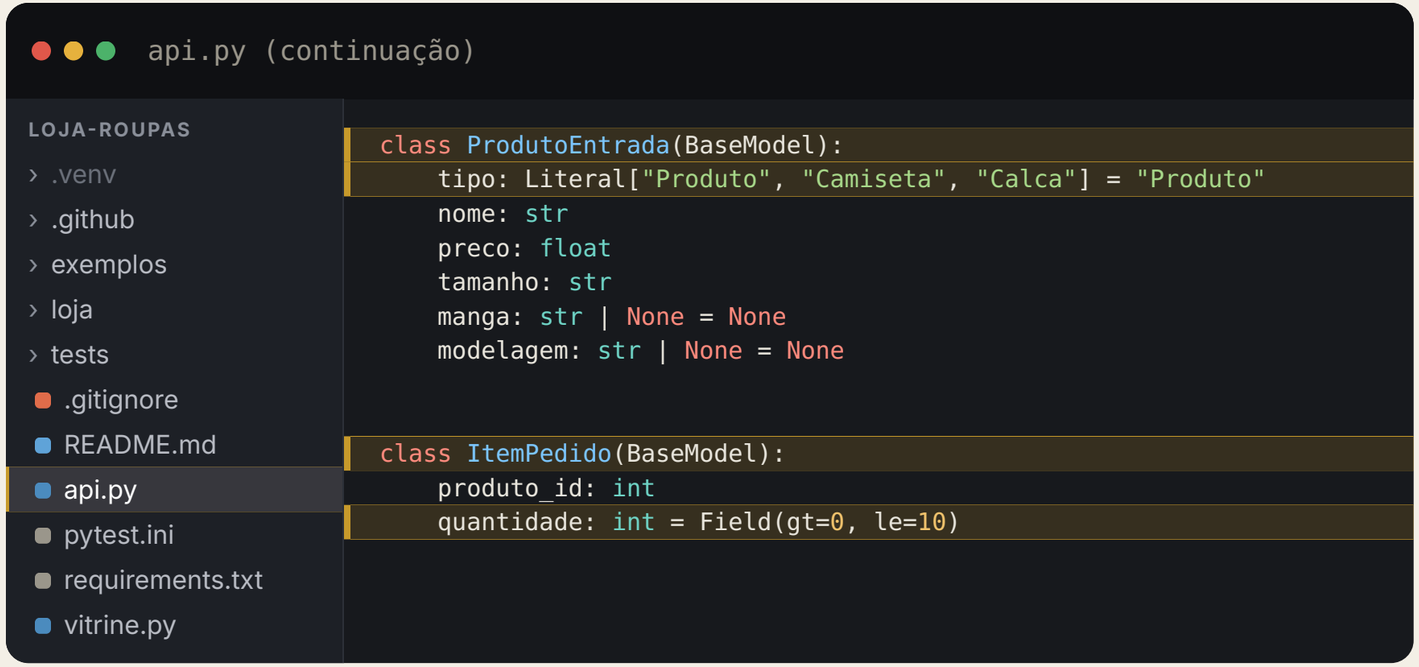
from fastapi import Depends, FastAPI
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field
from sqlmodel import Session

from loja.banco import criar_engine, criar_tabelas
from loja.carrinho import Carrinho
from loja.promocao import Percentual
from loja.repositorio import RepositorioProdutos
from loja.servico import (DadosInvalidos, ProdutoJaExiste, ProdutoNaoEncontrado,
                          ServicoCatalogo, com_id)
```

- A rota importa o **serviço**, as exceções do negócio e o banco, e nenhuma regra.
- `com_id` é o mesmo que o serviço usa para devolver o produto com o id.
- `Depends` e `Session` entram na injeção da sessão, logo adiante.

FASE 3-3 · NO EDITOR

api.py: os formatos de entrada



- `ProdutoEntrada` é o JSON que o cliente manda no POST e no PUT.
- O Pydantic recusa com **422** o que não bate com esse formato.
- `ItemPedido` é o do carrinho, que continua como na Aula 9.

FASE 3-3 · NO EDITOR

api.py: ligar o banco quando o servidor liga

api.py

```
engine = criar_engine()

CATALOGO_INICIAL = [
    {"tipo": "Camiseta", "nome": "Camiseta básica", "preco": 39.90, "tamanho": "M", "manga": "curta"},
    {"tipo": "Calca", "nome": "Calça jeans", "preco": 129.90, "tamanho": "G", "modelagem": "reta"},
    {"tipo": "Produto", "nome": "Moletom", "preco": 159.90, "tamanho": "P"},
]


def popular_se_vazio(servico):
    if not servico.listar():
        for dados in CATALOGO_INICIAL:
            servico.cadastrar(dados)


@asynccontextmanager
async def ciclo_de_vida(app):
    criar_tabelas(engine)
    with Session(engine) as sessao:
        popular_se_vazio(ServicoCatalogo(RepositorioProdutos(sessao)))
    yield


app = FastAPI(title="Loja de Roupas", lifespan=ciclo_de_vida)
```

- O `ciclo_de_vida` roda uma vez, quando o servidor liga: cria a tabela e coloca os 3 produtos se estiver vazia.
- Na segunda vez, o banco já tem dados, e nada é repetido.

FASE 3-3 · NO EDITOR

api.py: dependências e tradução dos erros

api.py (continuação)

```python
def obter_sessao():
    with Session(engine) as sessao:
        yield sessao


def obter_servico(sessao: Session = Depends(obter_sessao)):
    return ServicoCatalogo(RepositorioProdutos(sessao))


# ------------------------------------------------ exceção do domínio -> código HTTP
@app.exception_handler(ProdutoNaoEncontrado)
def nao_encontrado(request, erro):
    return JSONResponse(status_code=404, content={"detail": str(erro)})


@app.exception_handler(ProdutoJaExiste)
def ja_existe(request, erro):
    return JSONResponse(status_code=409, content={"detail": str(erro)})


@app.exception_handler(DadosInvalidos)
def dados_invalidos(request, erro):
    return JSONResponse(status_code=422, content={"detail": str(erro)})
```

- `yield` entrega a sessão para a rota e fecha quando o pedido termina.
- Os `exception_handler` traduzem num lugar só: nenhuma rota precisa de `try`.

FASE 3-3 · NO EDITOR

api.py: ler e cadastrar

api.py (continuação)

LOJA-ROUPAS

- .venv
- .github
- exemplos
- loja
- tests
- .gitignore
- README.md
- api.py
- pytest.ini
- requirements.txt
- vitrine.py

```python
@app.get("/produtos")
def listar_produtos(tamanho: str | None = None,
                    preco_max: float | None = None,
                    servico=Depends(obter_servico)):
    return servico.listar(tamanho, preco_max)


@app.get("/produtos/{produto_id}")
def buscar_produto(produto_id: int, servico=Depends(obter_servico)):
    return servico.buscar(produto_id)


@app.post("/produtos", status_code=201)
def cadastrar_produto(dados: ProdutoEntrada,
                      servico=Depends(obter_servico)):
    return servico.cadastrar(dados.model_dump())
```

- Cada rota tem **uma linha** de trabalho: chama o serviço.
- `model_dump()` transforma o modelo Pydantic em dicionário para o serviço.

FASE 3-3 · NO EDITOR

api.py: alterar e remover



- `PUT` manda o produto inteiro, já com as mudanças.
- `DELETE` responde **204**: removido, sem corpo para devolver.
- O carrinho continua na memória: persistir pedidos fica como desafio.

FASE 3-3 · NA /DOCS

O catálogo completo, gravado no banco



- **C**reate: POST, 201.
- **R**ead: GET, 200 ou 404.
- **U**pdate: PUT, 200, 404 ou 409.
- **D**elete: DELETE, 204 ou 404.

FASE 3-3 · NO TERMINAL

A prova: desligar e ligar de novo

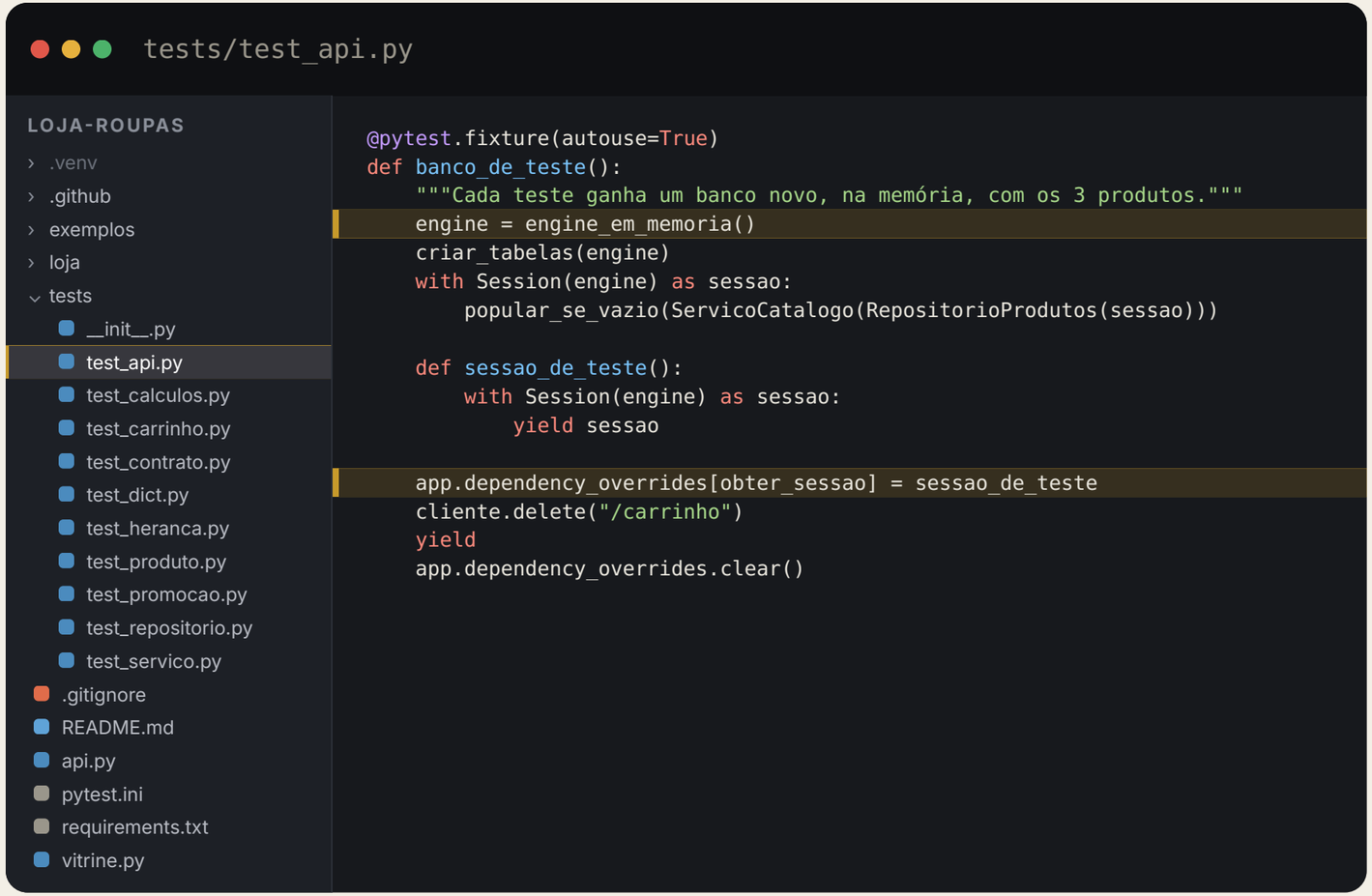
terminal

```
$ fastapi dev api.py
# em outro terminal:
$ curl -X POST $API/produtos -H "$JSON" -d '{"nome": "Boné", "preco": 49.9, "tamanho": "M"}'
{"id":4,"tipo":"Produto","nome":"Boné",...}
$ curl -i -X POST $API/produtos -H "$JSON" -d '{"nome": "Boné", "preco": 49.9, "tamanho": "M"}'
HTTP/1.1 409 Conflict
# Ctrl+C no servidor e fastapi dev api.py de novo
$ curl $API/produtos/4
{"id":4,"nome":"Boné",...}
```

- O Boné **sobreviveu** ao reinício: está no `loja.db`.
- Repetir o cadastro dá **409**, não um erro 500.
- Mais fácil pela `/docs`: o resultado é o mesmo.

FASE 3-3 · NO EDITOR

tests/test_api.py: cada teste com o próprio banco



- `dependency_overrides` troca o banco de verdade pelo de memória: o `loja.db` nunca é tocado nos testes.
- Os 7 testes da Aula 9 continuam, e entram 5 novos: cadastro, 409, 422, alteração e remoção.

FASE 3-3 · CONCLUÍDA

Checkpoint: salvar e subir

checkpoint

```
$ pytest
58 passed
$ git add .
$ git commit -m "Fase 3-3: API com CRUD em camadas"
$ git push
```

CONFIRA

O `pytest` mostra **58 passed**: os 53 de antes e 5 testes novos da API. O servidor reinicia sem perder o Boné e o GitHub Actions fica verde.

TRAVOU NESTA FASE?

Baixe a fase pronta e copie os arquivos para a sua pasta:

`github.com/sebastrogers/loja-roupas-gabarito/archive/refs/tags/etapa-15.zip`

ou, no terminal: `git clone https://github.com/sebastrogers/loja-roupas-gabarito` e `git checkout etapa-15`

LABORATÓRIO DA EQUIPE

Um recurso do projeto, gravado no banco

O QUE FAZER

CRUD em camadas

1. Uma tabela com SQLModel.
2. Um repositório com os 5 métodos.
3. Um serviço com pelo menos **uma regra** e as exceções.
4. As rotas `GET`, `POST`, `PUT` e `DELETE`.

CRITÉRIO DE PRONTO

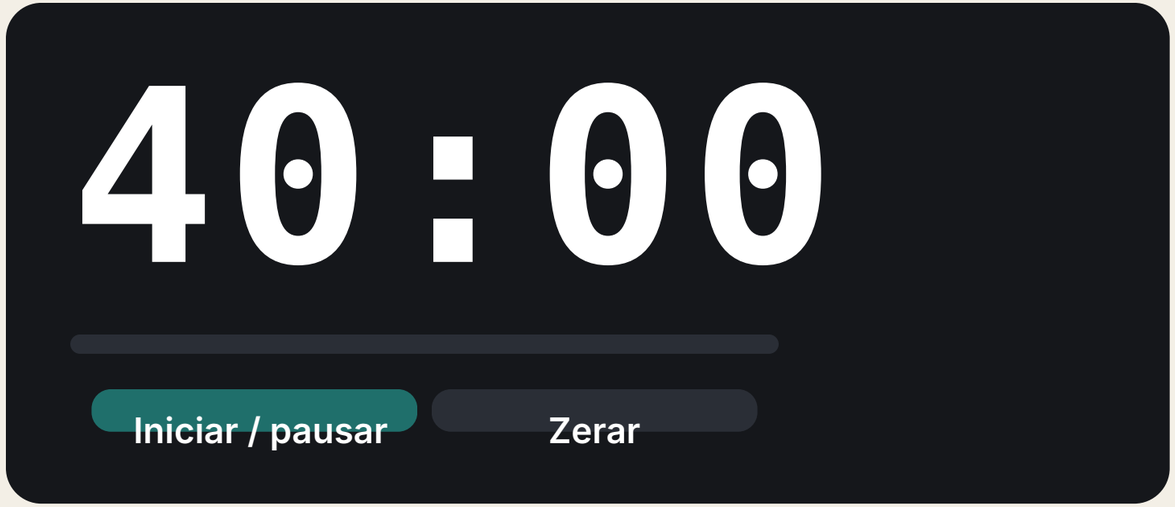
Para considerar entregue

Dados que sobrevivem ao reinício.
Testes do serviço e da API com **banco em memória**.
Códigos 201, 204, 404, 409 e 422 nos lugares certos.
Push feito e Actions verde.

Comecem pelo recurso principal do domínio de vocês. A loja da etapa-15 é o modelo: troquem Produto pela classe da equipe.

LABORATÓRIO DA EQUIPE

Mãos à obra

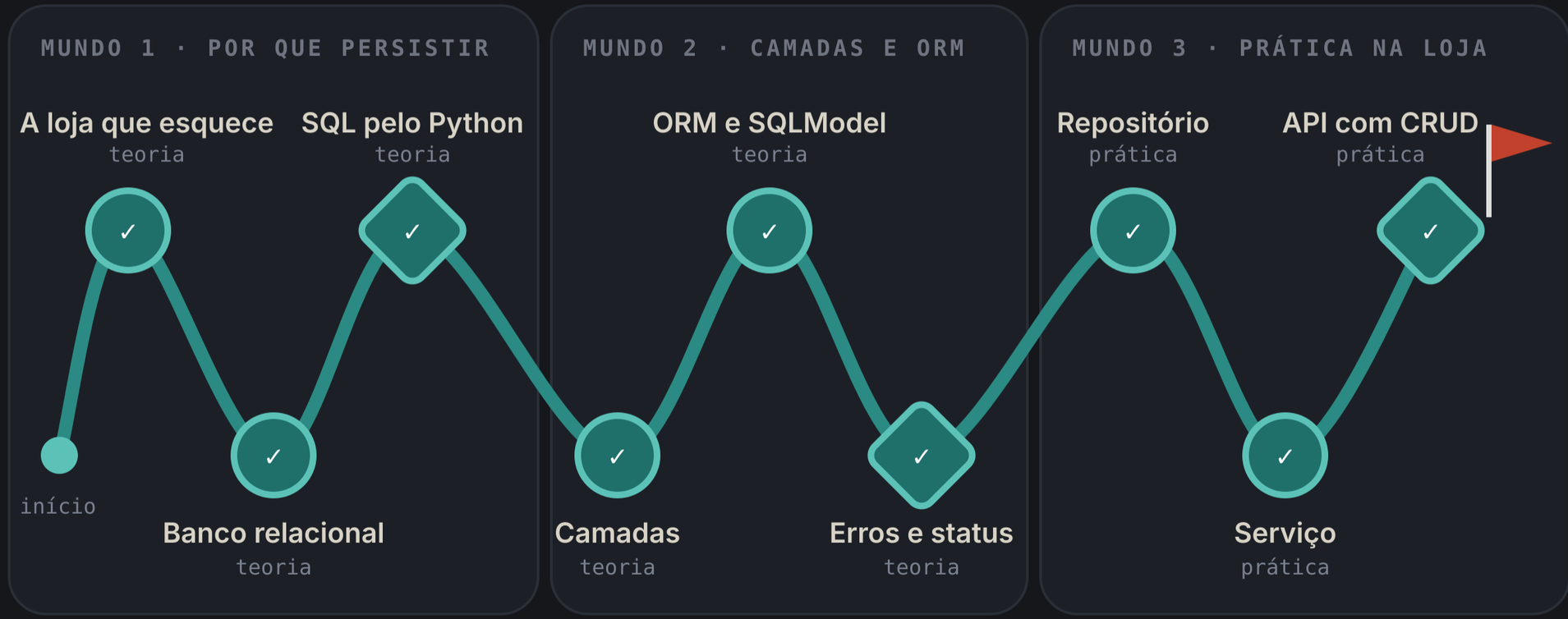


- Instalar o SQLModel e atualizar `requirements.txt` e `.gitignore`.
- Criar a tabela e o repositório, com um teste.
- Criar o serviço com uma regra e as exceções.
- Ligar as rotas ao serviço e traduzir os erros.
- Rodar o `pytest`, commit e push.

ENTREGA 1 · TERÇA, 27/10

Checklist do que a equipe apresenta

ITEM	COMO A EQUIPE MOSTRA
API do domínio	a página `/docs` com os endpoints do recurso principal
Dados guardados	cadastrar, reiniciar o servidor, e o dado continua lá
Camadas	rota, serviço e repositório em arquivos separados
Testes	`pytest` verde e o ✓ do GitHub Actions
README	o que é o sistema e como rodar, do `git clone` ao `fastapi dev`



O MAPA DE HOJE

Mapa completo

A loja agora não esquece, e o código está dividido em camadas que aguentam crescer.

ONDE ESTAMOS AGORA

O back-end está completo

Telas no navegador	o que o cliente vê e clica · Aulas 11 a 13	falta
▲ fetch ▼		
API: HTTP + REST	a porta da loja para a rede · Aula 9	✔ pronto
▲ chama ▼		
Serviço e repositório	as camadas entre a porta e o banco · Aula 10	✔ pronto
▲ usa ▼		
Domínio: as classes	Produto, Carrinho, Promocao · Aulas 5 a 8	✔ pronto
▲ guarda ▼		
Banco de dados	os dados deixam de sumir · Aula 10	✔ pronto

- O back-end da loja está **pronto**: API, regras e banco.
- Na **Aula 11**: a Entrega 1 e o começo do front-end, com HTML e CSS.
- Na **Aula 13**, as telas vão chamar esta API.

FECHAMENTO

O que cada fase construiu

FASE	CONCEITO	O QUE PROVA
1-1	persistência, memória volátil, dado derivado	o que vai para o banco
1-2	tabela, chave primária, UNIQUE, SQLite	SQLite x MySQL
1-3	conexão, cursor, commit, SQL injection	o commit que faltava
2-1	camadas, repositório, serviço	quem sabe o quê
2-2	ORM, SQLModel, sessão	trocar de banco numa linha
2-3	exceção do domínio, Depends, 404, 409, 422	o diagnóstico dos quatro casos
3-1	repositório na loja	`test_dados_sobrevivem_ao_reinicio`
3-2	serviço na loja	`test_nome_repetido_nao_entra`
3-3	API com CRUD e banco de teste	58 passed e o Boné depois do reinício

O catálogo agora é persistente. O carrinho continua na memória de propósito: persistir os pedidos é o desafio da equipe.

PARA ESTUDAR

Referências

BÁSICAS

ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2018.

PYTHON. *sqlite3: interface DB-API 2.0 para bancos de dados SQLite*. Documentação oficial.
https://docs.python.org/pt-br/3/library/sqlite3.html

FASTAPI. *Bancos de dados SQL*. Tutorial oficial.
https://fastapi.tiangolo.com/pt/tutorial/sql-databases/

FASTAPI. *Dependências*. Tutorial oficial.
https://fastapi.tiangolo.com/pt/tutorial/dependencies/

SQLMODEL. *Documentação*.
https://sqlmodel.tiangolo.com/

COMPLEMENTARES

FOWLER, M. *Padrões de arquitetura de aplicações corporativas*. Porto Alegre: Bookman, 2006. Repository e Service Layer.
https://martinfowler.com/eaaCatalog/repository.html

SQLITE. *Appropriate Uses For SQLite*.
https://www.sqlite.org/whentouse.html

OWASP. *SQL Injection*.
https://owasp.org/www-community/attacks/SQL_Injection

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. *RFC 9110: HTTP Semantics*, seção 15.5.10 (409 Conflict). IETF, 2022.
https://www.rfc-editor.org/rfc/rfc9110

PARA DEIXAR ABERTO

Cola da aula

sqlmodel

```python
# tabela
class X(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
# sessão
with Session(engine) as s:
    s.add(obj); s.commit(); s.refresh(obj)
    s.get(X, 1)
    s.exec(select(X).where(X.campo == v)).all()
    s.delete(obj); s.commit()
```

códigos

```
201  criado (POST)
204  removido, sem corpo (DELETE)
404  não existe          ProdutoNaoEncontrado
409  conflito            ProdutoJaExiste
422  dado recusado       DadosInvalidos / Pydantic
```